

CAP 4611 Assignment 2

Important: Submission Format [5 points]

Please make sure to follow the submission instructions posted on the course website and on the webcourses. Here is a direct link to the instructions. [We will deduct marks if the submission format is incorrect, or if your handwriting is *at all* difficult to read](#) – at least these 5 points, more for egregious issues. Be sure to include your name and student number. If you are working in a group of two, include both names and student numbers and only upload one submission. For more detailed instructions, see the link above.

1 K-Nearest Neighbours [15 points]

In the *citiesSmall* dataset (introduced in A1), nearby points tend to receive the same class label because they are part of the same U.S. state. For this problem, perhaps a k -nearest neighbours classifier might be a better choice than a decision tree. The file *knn.py* has implemented the training function for a k -nearest neighbour classifier (which is to just memorize the data).

Fill in the `predict` function in *knn.py* so that the model file implements the k -nearest neighbour prediction rule. You should use Euclidean distance, and may find numpy's `sort` and/or `argsort` functions useful. You can also use `utils.euclidean_dist_squared`, which computes the squared Euclidean distances between all pairs of points in two matrices. Also, please note that you will have to write code in the *main.py* file in order to do parts 2 and 3 of this question.

1. Write the `predict` function. Note that you can use the `utils.mode()` function in your implementation, and you can rely on this function to correctly handle potential ties when assigning a label to a training data point. [Submit this code.](#) [5 points]

Answer:

```
def predict(self, X_hat):
    distances = euclidean_dist_squared(self.X, X_hat)

    numTest = X_hat.shape[0]
    y_hat = np.zeros(numTest, dtype=self.y.dtype)

    for i in range(numTest):
        indices = np.argsort(distances[:, i])[:self.k]
        y_hat[i] = utils.mode(self.y[indices])

    return y_hat
```

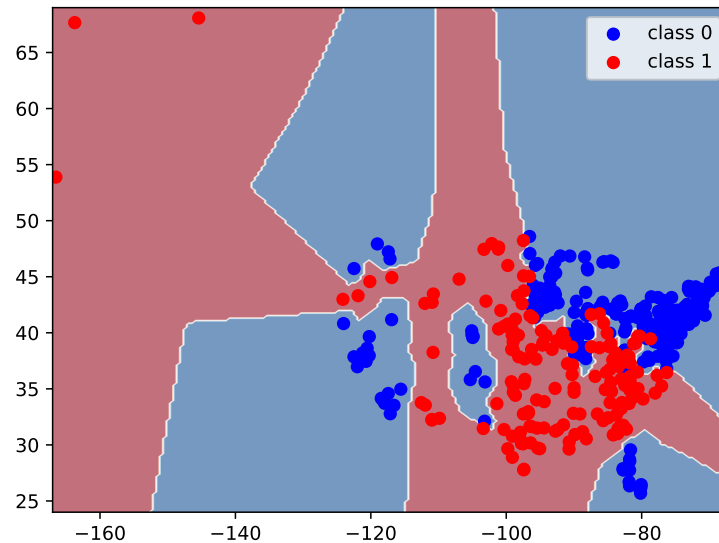
2. [Report the training and test error](#) obtained on the *citiesSmall* dataset for $k = 1$, $k = 3$, and $k = 10$. *Optionally*, try running a decision tree on this same train/test split; which gets better test accuracy? [4 points]

Answer:

$k = 1$: Training Error = 0.0, Test Error = 0.065

$k = 3$: Training Error = 0.028, Test Error = 0.066
 $k = 10$: Training Error = 0.058, Test Error = 0.085

3. Generate a plot with `utils.plot_classifier` on the *citiesSmall* dataset (plotting the training points) for $k = 1$, using your implementation of kNN. [Include the plot here](#). To see if your implementation makes sense, you might want to check against the plot using `sklearn.neighbors.KNeighborsClassifier`. Remember that the assignment 1 code had examples of plotting with this function and saving the result, if that would be helpful. [2 points]



4. Why is the training error 0 for $k = 1$? [2 points]

Answer: Training error is 0 for $k = 1$ because the nearest neighbor to each training example is itself, so each example is given the correct label.

5. Recall that we want to choose hyper-parameters so that the test error is (hopefully) minimized. How would you choose k ? [2 points]

Answer: With cross validation!

2 Picking k in kNN [15 points]

The file `data/ccdata.pkl` contains a subset of Statistics Canada’s 2019 Survey of Financial Security; we’re predicting whether a family regularly carries credit card debt, based on a bunch of demographic and financial information about them. (You might imagine social science researchers wanting to do something like this if they don’t have debt information available – or various companies wanting to do it for less altruistic reasons.) If you’re curious what the features are, you can look at the `'feat_descs'` entry in the dataset dictionary.

Anyway, now that we have our kNN algorithm working,¹ let’s try choosing k on this data!

1. Remember the golden rule: we don’t want to look at the test data when we’re picking k . Inside the `q2()` function of `main.py`, implement 10-fold cross-validation, evaluating on the `ks` set there (1, 5, 9, ..., 29), and store the *mean* accuracy across folds for each k into a variable named `cv_accs`.

Specifically, make sure you test on the first 10% of the data after training on the remaining 90%, then test on 10% to 20% and train on the remainder, etc – don’t shuffle (so your results are consistent with ours; the data is already in random order). Implement this yourself, don’t use scikit-learn or any other existing implementation of splitting. There are lots of ways you could do this, but one reasonably convenient way is to create a numpy “mask” array, maybe using `np.ones(n, dtype=bool)` for an all-`True` array of length `n`, and then setting the relevant entries to `False`. It also might be helpful to know that `~ary` flips a boolean array (`True` to `False` and vice-versa).

[Submit this code](#), following the general submission instructions to include your code in your results file. [5 points]

Answer:

```
def q2():
    dataset = load_dataset('ccdebt.pkl')
    X = dataset['X']
    y = dataset['y']
    X_test = dataset['Xtest']
    y_test = dataset['ytest']

    ks = list(range(1, 30, 4))

    n = X.shape[0]
    n_folds = 10
    fold_size = n // n_folds

    cv_accs = np.zeros(len(ks))
    test_accs = np.zeros(len(ks))
    train_errs = np.zeros(len(ks))

    for index, k in enumerate(ks):
        fold_accs = np.zeros(n_folds)

        for fold in range(n_folds):
            mask = np.ones(n, dtype=bool)

            start = fold * fold_size
            end = start + fold_size
```

¹If you haven’t finished the code for question 1, or if you’d just prefer a slightly faster implementation, you can use scikit-learn’s `KNeighborsClassifier` instead. The `fit` and `predict` methods are the same; the only difference for our purposes is that `KNN(k=3)` becomes `KNeighborsClassifier(n_neighbors=3)`.

```

mask[start:end] = False #False = Test

X_train_fold = X[mask]
y_train_fold = y[mask]
X_validation = X[~mask]
y_validation = y[~mask]

model = KNN(k)
model.fit(X_train_fold, y_train_fold)

y_hat_fold = model.predict(X_validation)
fold_accs[fold] = np.mean(y_hat_fold == y_validation)

cv_accs[index] = np.mean(fold_accs)

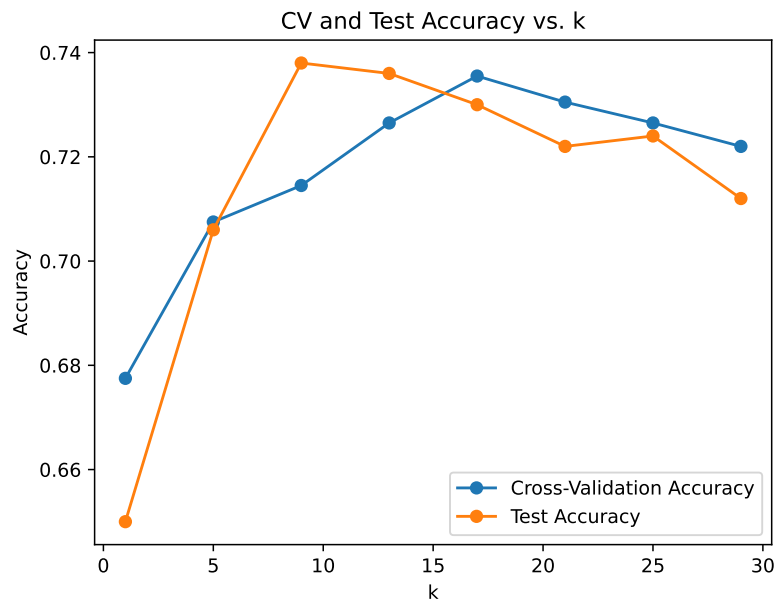
model = KNN(k)
model.fit(X, y)

y_hat_train = model.predict(X)
train_errs[index] = np.mean(y_hat_train != y)

y_hat_test = model.predict(X_test)
test_accs[index] = np.mean(y_hat_test == y_test)

```

2. The point of cross-validation is to get a sense of what the test error for a particular value of k would be. Implement, similarly to the code you wrote for question 1.2, a loop to compute the test accuracy for each value of k above. [Submit a plot the cross-validation and test accuracies as a function of \$k\$.](#) Make sure your plot has axis labels and a legend. [5 points]

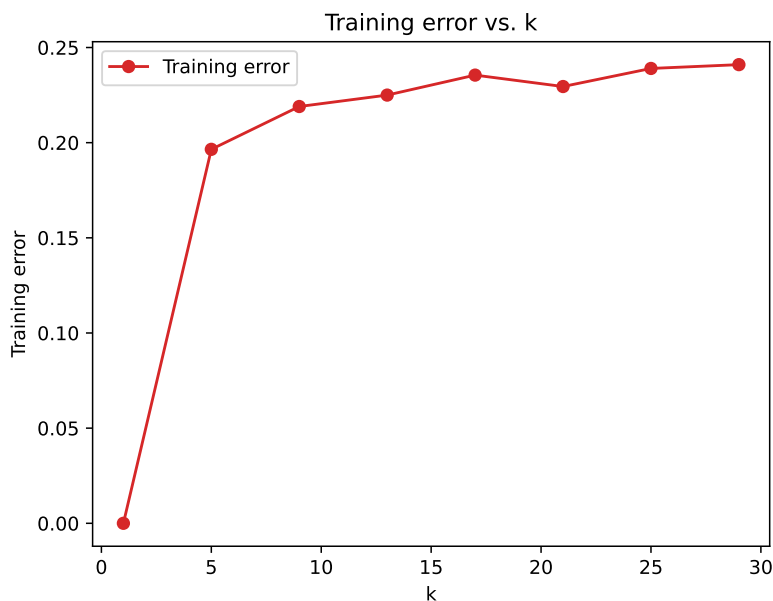


3. Which k would cross-validation choose in this case? Which k has the best test accuracy? Would the

cross-validation k do okay (qualitatively) in terms of test accuracy? [2 points]

Answer: Cross validation would choose $k = 17$, while $k = 9$ has the best test accuracy. Qualitatively yes, the cross-validation k does okay in terms of test accuracy, peaking a little after test accuracy does.

4. Separately, submit a plot of the training error as a function of k . How would the k with best training error do in terms of test error, qualitatively? [3 points]



Answer: The k with the best training error is $k = 1$, with its error being 0, but in terms of test error it performs the worst, since test accuracy is at its lowest when $k = 1$.

3 Naïve Bayes [17 points]

In this section we'll implement Naïve Bayes, a very fast classification method that is often surprisingly accurate for text data with simple representations like bag of words.

3.1 Naïve Bayes by Hand [5 points]

Consider the dataset below, which has 10 training examples and 3 features:

$$X = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad y = \begin{bmatrix} \text{spam} \\ \text{spam} \\ \text{spam} \\ \text{spam} \\ \text{spam} \\ \text{spam} \\ \text{not spam} \\ \text{not spam} \\ \text{not spam} \\ \text{not spam} \end{bmatrix}.$$

The feature in the first column is <your name> (whether the e-mail contained your name), in the second column is “lottery” (whether the e-mail contained this word), and the third column is “Venmo” (whether the e-mail contained this word). Suppose you believe that a naive Bayes model would be appropriate for this dataset, and you want to classify the following test example:

$$\hat{x} = [1 \quad 1 \quad 0].$$

3.1.1 Prior probabilities [1 points]

Compute the estimates of the class prior probabilities, which I also called the “baseline spam-ness” in class. (you don't need to show any work):

- $\Pr(\text{spam})$.

Answer: $\frac{6}{10}$

- $\Pr(\text{not spam})$.

Answer: $\frac{4}{10}$

3.1.2 Conditional probabilities [1 points]

Compute the estimates of the 6 conditional probabilities required by Naïve Bayes for this example (you don't need to show any work):

- $\Pr(\text{<your name>} = 1 \mid \text{spam})$.

Answer: $\frac{1}{6}$

- $\Pr(\text{lottery} = 1 \mid \text{spam})$.

Answer: $\frac{5}{6}$

- $\Pr(\text{Venmo} = 0 \mid \text{spam})$.

Answer: $\frac{2}{6}$

- $\Pr(\text{<your name>} = 1 \mid \text{not spam})$.

Answer: $\frac{3}{4}$

- $\Pr(\text{lottery} = 1 \mid \text{not spam})$.

Answer: $\frac{1}{4}$

- $\Pr(\text{Venmo} = 0 \mid \text{not spam})$.

Answer: $\frac{3}{4}$

3.1.3 Prediction [2 points]

Under the naive Bayes model and your estimates of the above probabilities, what is the most likely label for the test example? (Show your work.)

Answer:

$$p(y = \text{spam} \mid \hat{x}) = \frac{1}{6} * \frac{5}{6} * \frac{2}{6} * \frac{6}{10} = 0.0278$$

$$p(y = \text{notspam} \mid \hat{x}) = \frac{3}{4} * \frac{1}{4} * \frac{3}{4} * \frac{4}{10} = 0.0563$$

Not spam is the most likely label for the test example.

3.1.4 Simulating Laplace Smoothing with Data [1 points]

One way to think of Laplace smoothing is that you're augmenting the training set with extra counts. Consider the estimates of the conditional probabilities in this dataset when we use Laplace smoothing (with $\beta = 1$). Give a set of extra training examples where, if they were included in the training set, the "plain" estimation method (with no Laplace smoothing) would give the same estimates of the conditional probabilities as using the original dataset with Laplace smoothing. Present your answer in a reasonably easy-to-read format, for example the same format as the data set at the start of this question.

Answer:

$$X = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}, \quad y = \begin{bmatrix} \text{spam} \\ \text{spam} \\ \text{not spam} \\ \text{not spam} \end{bmatrix}$$

3.2 Exploring Bag-of-Words [2 points]

If you run `python main.py 3.2`, it will load the following dataset:

1. **X**: A binary matrix. Each row corresponds to a newsgroup post, and each column corresponds to whether a particular word was used in the post. A value of 1 means that the word occurred in the post.
2. **wordlist**: The set of words that correspond to each column.
3. **y**: A vector with values 0 through 3, with the value corresponding to the newsgroup that the post came from.
4. **groupnames**: The names of the four newsgroups.
5. **Xvalidate** and **yvalidate**: the word lists and newsgroup labels for additional newsgroup posts.

Answer the following:

1. Which word corresponds to column 73 of *X*? (This is index 72 in Python.)

Answer: The word is "question."

2. Which words are present in training example 803 (Python index 802)?

Answer: The words present are "case," "children," "health," "help," "problem," and "program."

3. Which newsgroup name does training example 803 come from?

Answer: Example 803 came from newsgroup "talk."

3.3 Naïve Bayes Implementation [4 points]

If you run `python main.py 3.3` it will load the newsgroups dataset, fit a basic naive Bayes model and report the validation error.

The `predict()` function of the naive Bayes classifier is already implemented. However, in `fit()` the calculation of the variable `p_xy` is incorrect (right now, it just sets all values to 1/2). [Modify this function so that `p\_xy` correctly computes the conditional probabilities of these values based on the frequencies in the data set. Submit your code. Report the training and validation errors that you obtain.](#)

Answer:

```
def fit(self, X, y):
    n, d = X.shape

    # Compute the number of class labels
    k = self.num_classes

    # Compute the probability of each class i.e p(y=c), aka 'baseline -ness'
    counts = np.bincount(y)
    p_y = counts / n

    p_xy = 0.5 * np.zeros((d, k))

    for c in range(k):
        X_c = X[y == c]
        p_xy[:, c] = X_c.mean(axis = 0)

    self.p_y = p_y
    self.p_xy = p_xy
```

Training Error: 0.200

Validation Error: 0.188

3.4 Laplace Smoothing Implementation [4 points]

Laplace smoothing is one way to prevent failure cases of Naïve Bayes based on counting. Recall what you know from lecture to implement Laplace smoothing to your Naïve Bayes model.

- Modify the `NaiveBayesLaplace` class provided in `naive_bayes.py` and write its `fit()` method to implement Laplace smoothing. [Submit this code](#).

Answer:

```
def fit(self, X, y):
    n, d = X.shape
    k = self.num_classes

    counts = np.bincount(y)
    p_y = counts / n

    p_xy = np.zeros((d, k))

    for c in range(k):
        X_c = X[y == c]
        n_c = X_c.shape[0]

        #Smoothing
        p_xy[:, c] = (X_c.sum(axis = 0) + self.beta) / (n_c + 2 * self.beta)

    self.p_y = p_y
    self.p_xy = p_xy
```

- Using the same data as the previous section, fit Naïve Bayes models with **and** without Laplace smoothing to the training data. Use $\beta = 1$ for Laplace smoothing. For each model, look at $p(x_{ij} = 1 \mid y_i = 0)$ across all j values (i.e. all features) in both models. [Do you notice any difference? Explain](#).

Answer: The difference between having and not having Laplace smoothing is that the words that don't have any examples marked as spam have a probability of zero without smoothing, but with smoothing they have a small, but nonzero, probability.

- One more time, fit a Naïve Bayes model with Laplace smoothing using $\beta = 10000$. Look at $p(x_{ij} = 1 \mid y_i = 0)$. [Do these numbers look like what you expect? Explain](#).

Answer: All the probabilities are close to 0.5, which is what we expect, since a larger β leads to more smoothing.

3.5 Runtime of Naïve Bayes for Discrete Data [2 points]

For a given training example i , the predict function in the provided code computes the quantity

$$p(y_i | x_i) \propto p(y_i) \prod_{j=1}^d p(x_{ij} | y_i),$$

for each class y_i (and where the proportionality constant is not relevant). For many problems, a lot of the $p(x_{ij} | y_i)$ values may be very small. This can cause the above product to underflow. The standard fix for this is to compute the logarithm of this quantity and use that $\log(ab) = \log(a) + \log(b)$,

$$\log p(y_i | x_i) = \log p(y_i) + \sum_{j=1}^d \log p(x_{ij} | y_i) + (\text{log of the irrelevant proportionality constant}).$$

This turns the multiplications into additions and thus typically would not underflow.

Assume you have the following setup:

- The training set has n objects each with d features.
- The test set has t objects with d features.
- Each feature can have up to c discrete values (you can assume $c \leq n$).
- There are k class labels (you can assume $k \leq n$).

You can implement the training phase of a naive Bayes classifier in this setup in $O(nd)$, since you only need to do a constant amount of work for each x_{ij} value. (You do not have to actually implement it in this way for the previous question, but you should think about how this could be done.) [What is the cost of classifying \$t\$ test examples with the model and this way of computing the predictions?](#)

Answer: For each test example, we'd need to compute the $\log p(y_i | x_i)$ for each of the t test examples, so the cost would be $O(tdk)$.

4 Random Forests [15 points]

The file `vowels.pkl` contains a supervised learning dataset where we are trying to predict which of the 11 “steady-state” English vowels that a speaker is trying to pronounce.

You are provided with a `RandomStump` class that differs from `DecisionStumpInfoGain` in that it only considers $\lfloor \sqrt{d} \rfloor$ randomly-chosen features.² You are also provided with a `RandomTree` class that is exactly the same as `DecisionTree` except that it uses `RandomStump` instead of `DecisionStump` and it takes a bootstrap sample of the data before fitting. In other words, `RandomTree` is the entity we discussed in class, which makes up a random forest.

If you run `python main.py 4` it will fit a deep `DecisionTree` using the information gain splitting criterion. You will notice that the model overfits badly.

1. Using the provided code, evaluate the `RandomTree` model of unlimited depth. Why doesn't the random tree model have a training error of 0? [2 points]

Answer: Since the tree is trained on a bootstrap of the dataset, it (likely) doesn't have every training example, so it doesn't have perfect classification, even though it memorizes the data it was trained on.

2. For `RandomTree`, if you set the `max_depth` value to `np.inf`, why do the training functions terminate instead of making an infinite number of splitting rules? [2 points]

Answer: The training functions terminate since the set it is training on is finite, so it has to terminate at some point.

3. Complete the `RandomForest` class in `random_tree.py`. This class takes in hyperparameters `num_trees` and `max_depth` and fits `num_trees` random trees each with maximum depth `max_depth`. For prediction, have all trees predict and then take the mode. Submit this code. [5 points]

Answer:

```
class RandomForest:
    def __init__(self, num_trees, max_depth):
        self.num_trees = num_trees
        self.max_depth = max_depth
        self.trees = []

    def fit(self, X, y):
        self.trees = []
        for i in range(self.num_trees):
            tree = RandomTree(max_depth=self.max_depth)
            tree.fit(X, y)
            self.trees.append(tree)

    def predict(self, X_pred):
        n = X_pred.shape[0]
        tree_preds = np.zeros((self.num_trees, n), dtype=np.uint8)

        for i, tree in enumerate(self.trees):
            tree_preds[i, :] = tree.predict(X_pred)

        y_pred = np.zeros(n, dtype=np.uint8)
```

²The notation $\lfloor x \rfloor$ means the “floor” of x , or “ x rounded down”. You can compute this with `np.floor(x)` or `math.floor(x)`.

```

    for i in range(n):
        y_pred[i] = utils.mode(tree_preds[:, i])

    return y_pred

```

4. Using 50 trees, and a max depth of ∞ , **report the training and testing error**. Compare this to what we got with a single `DecisionTree` and with a single `RandomTree`. **Are the results what you expected? Discuss.** [3 points]

Answer: Forest training error: 0.00, Forest test error: 0.197

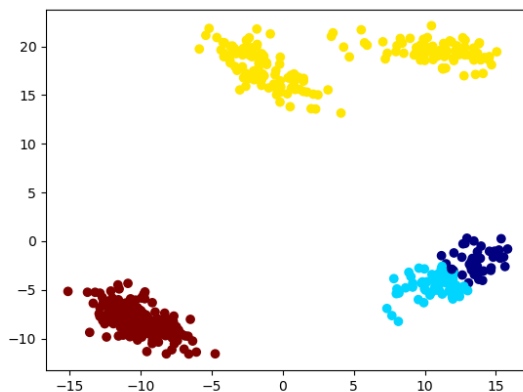
The forest has significantly lower errors than either tree, which is expected. The decision tree will be very overfitted, since it would be built to basically memorize the training data, and therefore have a bad test error. The random tree is even worse, since it would be built on a less representative version of the dataset the normal tree was built off of. The forest, because of having many random trees, achieves perfect training error, and much lower test error.

5. **Why does a random forest typically have a training error of 0, even though random trees typically have a training error greater than 0?** [3 points]

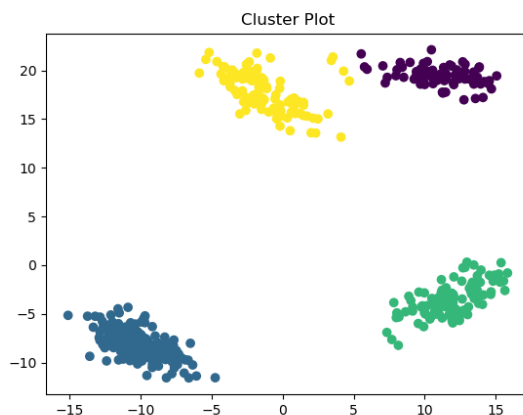
Answer: Because there will be enough trees that recieved a bootstrap with the training example to have the majority vote be the correct label.

5 Clustering [15 points]

If you run `python main.py 5`, it will load a dataset with two features and a very obvious clustering structure. It will then apply the k -means algorithm with a random initialization. The result of applying the algorithm will thus depend on the randomization, but a typical run might look like this:



(Note that the colours are arbitrary – this is the label switching issue.) But the “correct” clustering (that was used to make the data) is this:



5.1 Selecting Among k -means Initializations [7 points]

If you run the demo several times, it will find different clusterings. To select among clusterings for a *fixed* value of k , one strategy is to minimize the sum of squared distances between examples x_i and their means w_{y_i} ,

$$f(w_1, w_2, \dots, w_k, y_1, y_2, \dots, y_n) = \sum_{i=1}^n \|x_i - w_{y_i}\|_2^2 = \sum_{i=1}^n \sum_{j=1}^d (x_{ij} - w_{y_i j})^2.$$

where y_i is the index of the closest mean to x_i . This is a natural criterion because the steps of k -means alternately optimize this objective function in terms of the w_c and the y_i values.

1. In the `kmeans.py` file, complete the `error()` method. `error()` takes as input the data used in fit (`X`), the indices of each examples' nearest mean (`y`), and the current value of means (`means`). It returns the value of this above objective function. [Submit this code](#). What trend do you observe if you print the value of this error after each iteration of the k -means algorithm? [4 points]

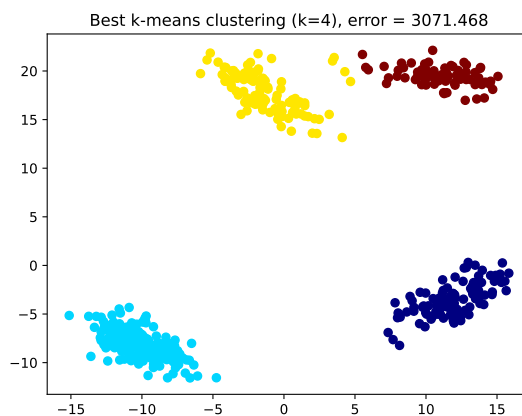
Answer:

```
def error(self, X, y, means):  
    return np.sum((X - means[y]) ** 2)
```

The error drops quickly in the first iteration then starts converging.

2. Run k -means 50 times (with $k = 4$) and take the one with the lowest error. [Report the lowest error obtained](#). Visualize the clustering obtained by this model, and [submit your plot](#). [3 points]

Answer: The lowest error = 3071.468



5.2 Selecting k in k -means [8 points]

We now turn to the task of choosing the number of clusters k .

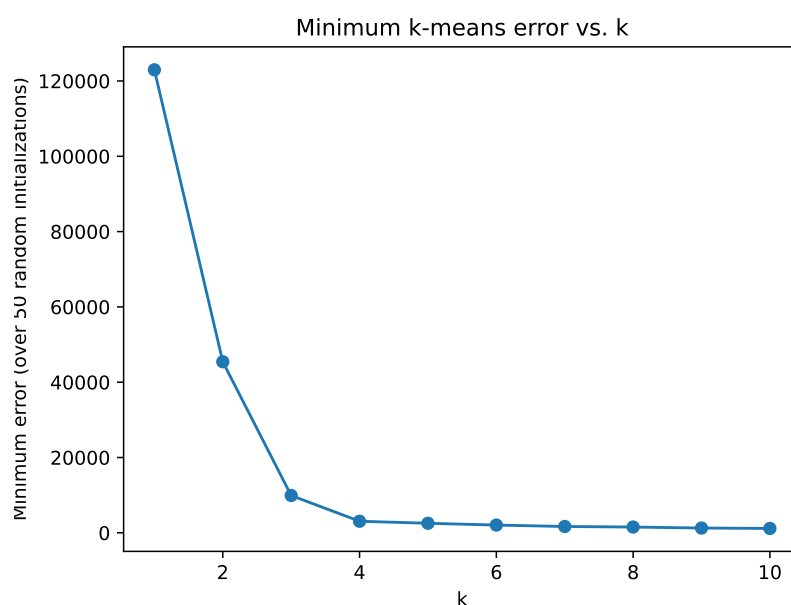
1. Explain why we should not choose k by taking the value that minimizes the error value. [2 points]

Answer: We shouldn't choose the k that would minimize the error value because it would make each point its own individual cluster, telling us nothing about the data. As k increases, error goes down. When $k = n$, error should equal 0, since each point is in its own cluster, so choosing $k = n$ would tell us nothing about the data.

2. Is evaluating the error function on validation (or test data) a suitable approach to choosing k ? [2 points]

Answer: No, because it would run into the same issue. Increasing k will always improve error, regardless of the data its measured on.

3. Hand in a plot of the minimum error found across 50 random initializations, as a function of k , taking k from 1 to 10. [2 points]



4. The *elbow method* for choosing k consists of looking at the above plot and visually trying to choose the k that makes the sharpest “elbow” (the biggest change in slope). What values of k might be reasonable according to this method? Note: there is not a single correct answer here; it is somewhat open to interpretation and there is a range of reasonable answers. [2 points]

Answer: The values of k most suitable with the elbow method would be either 3 or 4, as they produce the biggest change in slope.

6 Very-Short Answer Questions [18 points]

Write a short one or two sentence answer to each of the questions below. Make sure your answer is clear and concise.

1. What is a reason that the the data may not be IID in the email spam filtering example from lecture?

Answer: A reason the emails may not be IID is that the some of the emails would likely be about the same thing, making them not independent.

2. Why can't we (typically) use the training error to select a hyper-parameter?

Answer: Optimizing for training error would just lead the model to severely overfit, and therefore perform poorly on test sets.

3. What is the effect of the training or validation set size n on the optimization bias, assuming we use a parametric model?

Answer: As n increases, overfitting would decrease. This is because a larger n would be more representative of the underlying population.

4. What is an advantage and a disadvantage of using a large k value in k -fold cross-validation?

Answer: An advantage of a more folds is that each training phase is trained on a larger percentage of the data, while a disadvantage would be that it is more computationally expensive.

5. Recall that false positive in binary classification means $\hat{y}_i = 1$ while $\tilde{y}_i = 0$. Give an example of when increasing false positives is an acceptable risk.

Answer: Screening for cancers or other major diseases would be an acceptable time to increase false positives, since missing a true positive would be much costlier.

6. Why can we ignore $p(x_i)$ when we use naive Bayes?

Answer: Because $p(x_i)$ is constant for each class y .

7. For each of the three values below in a naive Bayes model, say whether it's better considered as a parameter or a hyper-parameter:

- (a) Our estimate of $p(y_i)$ for some y_i .

Answer: Parameter - it is not chosen before hand, the model estimates it.

- (b) Our estimate of $p(x_{ij} | y_i)$ for some x_{ij} and y_i .

Answer: Parameter - it is not chosen before hand, the model estimates it.

- (c) The value β in Laplace smoothing.

Answer: Hyper-parameter - it is chosen before hand.

8. Both supervised learning and clustering models take in an input x_i and produce a label y_i . What is the key difference between these types of models?

Answer: In supervised learning, the labels are meaningful and predefined. In clustering, the model creates the labels based on the structure of the data its provided.

9. In k -means clustering the clusters are guaranteed to be convex regions. Are the areas that are given the same label by kNN also convex?

Answer: No, since kNN can make disjointed shapes.